

BULUT MİMARİSİ VE TEST OTOMASYONU: QUOTE API PROJE RAPORU

1. Giriş

Projenin Amacı: Modern yazılım geliştirme süreçlerinde kod kalitesini korumak ve güvenilir sürümler çıkabilmek için "Sürekli Entegrasyon ve Sürekli Dağıtım" (CI/CD) hatları hayati öneme sahiptir. Bu projenin temel amacı; kodun yazılmasından canlı ortama alınmasına kadar geçen süreci (test, build, deploy) tamamen otomatize etmek, insan hatasını ortadan kaldırmak ve oluşturulan bulut mimarisini sürekli gözlemlenebilir (observable) kılmaktır.

Seçilen Mini-Uygulama: Proje konsepti olarak FastAPI tabanlı bir "Günün Sözü (Quote API)" uygulaması seçilmiştir. Bu uygulama, kullanıcıların alıntılarını/sözleri okuyabildiği, arayabildiği ve yeni sözler ekleyebildiği bir RESTful web servsidir. Sistem ayrıca, belirlenen günün sözünü AWS S3 bulut depolama servisine yedekleyecek şekilde kurgulanmıştır.

Motivasyon: Bu konunun seçilmesindeki ana motivasyon, basit bir uygulamanın arkasında karmaşık ve endüstri standardı bir altyapı (Kubernetes, AWS S3, Prometheus, Grafana, Playwright, k6) kurma pratiği yapmaktır. Uygulamanın iş mantığı kasıtlı olarak sade tutulmuş; eforun büyük kısmı test piramidinin inşasına, konteynerizasyon süreçlerine ve Kubernetes orkestrasyonuna harcanmıştır. "Benim makinemde çalışıyor" sorununu yok etmek ve bulut-yerli (cloud-native) bir sistemin nasıl yönetildiğini somut olarak deneyimlemek ana itici gücümüz olmuştur.

2. Mimari

Mimarimiz, mikroservis prensiplerine uygun olarak tasarlanmış ve dışa bağımlılıklar izole edilmiştir.

(Buraya [docs/architecture.png](#) dosyasındaki mimari diyagram eklenecektir) **[Mimari Diyagram - Görsel]**

Bileşenlerin Açıklaması:

- **Servis Katmanı (FastAPI):** Python 3.11 tabanlı, asenkron yapısıyla yüksek performans sunan FastAPI kullanılmıştır. Uvicorn sunucusu ile ayağa kalkar ve [quote-api-web](#) Docker imajı olarak paketlenir.
- **Veritabanı Katmanı (PostgreSQL 15):** Uygulamanın kalıcı verileri (sözler, yazarlar, kategoriler) ilişkisel bir veritabanı olan PostgreSQL üzerinde tutulmaktadır. Geliştirme ortamında Docker Compose, test ortamında Testcontainers, prod ortamında ise K8s Deployment üzerinden yönetilir.
- **AWS Simülasyonu (LocalStack):** Sistem, bulut bağımlılığı olarak AWS S3 kullanmaktadır. Ancak maliyet ve test izolasyonu sağlamak amacıyla, lokalde ve CI hattında AWS servislerini simüle eden **LocalStack** kullanılmıştır. Günün sözü [.txt](#) formatında [quote-backups](#) isimli bucket'a kaydedilmektedir.
- **Orkestrasyon (Kubernetes):** Tüm servisler Kubernetes (K8s) cluster'ı üzerinde çalışacak şekilde tasarlanmıştır. [k8s/](#) dizinindeki manifestlerle; uygulama için 2 replikalı bir Deployment, veritabanı için ayrı bir Pod ve dış dünya iletişimi için [NodePort](#) Service yapıları kurgulanmıştır. Ayrıca çevre değişkenleri [ConfigMap](#) üzerinden dinamik olarak enjekte edilmektedir.
- **Monitoring (Prometheus & Grafana):** Sistemin sağlığını izlemek için uygulamanın [/metrics](#) uç noktası [prometheus-fastapi-instrumentator](#) kütüphanesi ile dışa açılmıştır. Prometheus bu verileri kazır (scrape) ve Grafana bu verileri görselleştirir.

3. Test Stratejisi

Projede yüksek güvenilirlik sağlamak için endüstri standardı "Test Piramidi" modeli sıkı bir şekilde uygulanmıştır.

- **Birim (Unit) Testler (Piramidin Tabanı):** Servis iş mantığı (`src.services`), dış bağımlılıklardan (gerçek DB veya AWS) izole edilerek test edilmiştir. `pytest` kullanılarak yazılan bu testlerde, in-memory SQLite (veya mocked object'ler) kullanılarak testlerin milisaniyeler içinde koşması sağlanmıştır. (`test_services.py`)
- **Entegrasyon Testleri (Piramidin Ortası):** API'nin veritabanı ile gerçekten nasıl konuştuğunu görmek için `Testcontainers` kütüphanesi kullanılmıştır. Testler başladığında geçici bir PostgreSQL Docker konteyneri otomatik ayağa kalkar, API uç noktaları (`/quotes`, `/quotes/today` vb.) bu gerçek veritabanı üzerinde HTTP istekleri ile test edilir ve test bitiminde konteyner yok edilir. (`test_api.py`)
- **E2E (Uçtan Uca) ve Smoke Testler (Piramidin Tepesi):**
 - **Playwright:** Uygulamanın önyüzü (UI) üzerinden Chromium tarayıcısı simüle edilerek kullanıcı senaryoları (Söz ekleme, arama, butonlara tıklama) test edilmiştir. (`test_ui.py`)
 - **Postman/Newman:** CI hattının sonunda, sistemin temel endpoint'lerinin 200/404 durum kodlarını doğru dönüp dönmediğini kontrol eden duman (smoke) testleri koşulmuştur. (`collection.json`)
- **Coverage Hedefi:** CI hattında kod kapsama alanı (`--cov-fail-under=70`) %70 olarak zorunlu kılınmış, test yazılmamış kodların sisteme entegre olması engellenmiştir.

4. Pipeline & Deploy

CI/CD süreçleri, her kod push işleminde tetiklenen bir GitHub Actions iş akışı (`.github/workflows/ci.yml`) ile yönetilmektedir.

GitHub Actions Akışı:

1. **Statik Analiz (Lint):** Kod, PEP8 standartlarına uygunluğu açısından `Flake8` ile taranır.
2. **İzole Testler:** Unit ve entegrasyon testleri (Testcontainers destekli) Pytest üzerinden `--cov` parametresi ile çalıştırılır. Coverage %70'in altındaysa pipeline hata verir ve süreç durur (Fail-fast).
3. **Build İşlemi:** Testleri geçen kod, hafif bir Docker imajına (multi-stage build kullanarak) dönüştürülür ve commit hash'i ile etiketlenir (`quote-api-web:${{github.sha}}`).
4. **E2E Geçici Ortam (Ephemeral Environment):** Build edilen imaj, Docker Compose ile geçici bir şekilde (DB ve LocalStack dahil) ayağa kaldırılır. `Playwright` ve `Newman` uçtan uca testleri bu canlı-benzeri ortamda koşulur.
5. **Deploy Stratejisi (Kubernetes):** CI hattı başarıyla tamamlandıktan sonra imaj, K8s cluster'ına deploy edilir. Uygulamanın `k8s/deployment.yaml` dosyasında `replicas: 2` olarak ayarlandığı için "Rolling Update" stratejisi uygulanır. Yani eski pod'lar kapatılırken yeni versiyon pod'lar sırayla ayağa kalkar, böylece kullanıcılara sıfır kesinti (zero-downtime) yansıtılır.

5. Performans & Gözlemlenebilirlik

Sistemin yük altında nasıl davrandığını ve kaynak tüketimini anlık olarak görmek için K6 ve Grafana kullanılmıştır.

k6 Yük Testi Senaryosu: `perf/load-test.js` dosyası ile `GET /quotes/today` endpoint'ine bir yük testi uygulanmıştır. Test, 2 saniye içinde 20 sanal kullanıcıya (VU) tırmanır, 6 saniye boyunca bu yükü korur ve son 2 saniyede yükü sıfırlar.

- **Sonuç:** `report.md` çıktılarına göre başarı oranı %100'dür. Uygulamanın **p95 Latency (Gecikme) süresi 45ms** olarak ölçülmüştür. Bu, kullanıcıların %95'inin 45 milisaniyeden daha kısa sürede yanıt aldığını gösterir ve oldukça başarılı bir skordur.

Grafana Dashboard: Prometheus'un topladığı metrikler `monitoring/grafana-dashboard.json` şeması ile görselleştirilmiştir. Panellerde "Toplam İstek Sayısı (Total Requests)", "Hata Oranı (Error Rate 5xx)" ve "p95 Gecikme Süresi" anlık izlenmektedir.

(Buraya Grafana Dashboard Ekran Görüntüsü eklenecektir) **[Grafana Dashboard - Ekran Görüntüsü]**

6. Sonuç & Öğrendiklerim

Sayılarla Özet:

- Sistem uçtan uca çalışır durumdadır ve 4 farklı test aracı (Pytest, Testcontainers, Playwright, Newman) pipeline'a entegre edilmiştir.
- Yük altında sistem çökmemiş, p95 gecikme hedefi 50ms'nin altında tutulmuştur (45ms).
- Geliştirme, test ve dağıtım izolasyonu Docker ve Kubernetes sayesinde %100 güvence altına alınmıştır.

Karşılaşılan Zorluklar:

1. **Testcontainers ve Asenkron Servisler:** CI hattında veritabanı (PostgreSQL) konteynerinin ayağa kalkma süresi (startup time) ile API'nin bağlanma çabası arasında *Race Condition* (yarış durumu) sorunları yaşanmıştır. Bu sorun, Docker healthcheck mekanizmaları kurgulanarak çözüldü.
2. **Bulut Simülasyonu:** AWS S3'ü lokalde simüle eden LocalStack ile boto3 (Python AWS SDK) kütüphanesini bağlarken, endpoint tanımları ve kimlik doğrulama ayarlarını **mock** parametreleriyle ezmek, dokümantasyon eksikliğinden dolayı zorlayıcı bir süreçti.

İleride Yapılabilecekler:

- Mevcut yapı sadece CI (Continuous Integration) odaklıdır; K8s deployment'ı GitHub Actions içine tamamen entegre edilerek süreç tam bir CD (Continuous Deployment) haline getirilebilir.
- Gelecekte LocalStack yerine, Terraform kullanılarak doğrudan gerçek bir AWS (EKS, RDS, S3) ortamına geçiş yapılabilir (Infrastructure as Code).
- Kubernetes manifestlerini daha modüler yönetmek için **Hel**m paket yöneticisine veya **ArgoCD** ile GitOps prensiplerine geçilebilir.

8. Kaynaklar

1. FastAPI Resmi Dokümantasyonu: <https://fastapi.tiangolo.com/>
2. Testcontainers for Python: <https://testcontainers-python.readthedocs.io/>
3. Playwright Python Dokümantasyonu: <https://playwright.dev/python/>
4. k6 Load Testing Dokümantasyonu: <https://k6.io/docs/>
5. Kubernetes Pod & Deployment Dokümantasyonları: <https://kubernetes.io/docs/>
6. Prometheus & Grafana Entegrasyon Makaleleri (Çeşitli Medium yazıları ve StackOverflow tartışmaları)